

O uso de *logs* no auxílio de ensino de programação

1

Abstract. *In software development, it is common to use metrics to assess the quality of code and the time spent producing it. In Education is not default, however, this project will help to obtain new informations to recognize standard and metrics. This work proposes the creation of a dataset with records (logs) of the code development process, capturing the learning trail in real time. Unlike conventional approaches, which rely on Learning Management Systems, the collection will be carried out using a plugin integrated with a widely used code editor, aimed at both programming and documentation. The goal of this paper is to provide a detailed database for future analyses, contributing to the study of programming teaching and learning.*

Resumo. *No desenvolvimento de software, é comum utilizar métricas para avaliar a qualidade do código e o tempo despendido em sua produção. Este trabalho propõe a criação de um dataset com registros (logs) do processo de desenvolvimento de código, capturando o rastro da aprendizagem em tempo real. Diferentemente de abordagens convencionais, que dependem de Sistemas de Gestão de Aprendizagem, a coleta será realizada por meio de um plugin integrado a um editor de código amplamente utilizado, voltado tanto para programação quanto para documentação. Espera-se fornecer uma base de dados sobre o ensino-aprendizagem de programação que possa auxiliar na identificação de dificuldades dos estudantes, na personalização do ensino e no aprimoramento das práticas pedagógicas.*

1. Introdução

Enquanto o ensino tradicional se baseia na leitura e escrita de textos, cursos de programação introduzem uma linguagem singular: os algoritmos — sequências lógicas interpretáveis tanto por humanos quanto por máquinas. Durante seu desenvolvimento, geram-se registros automáticos (logs), que capturam desde erros de sintaxe até padrões de depuração. Esses dados têm potencial para transformar o ensino de programação, mas sua aplicação enfrenta desafios críticos: ferramentas existentes (como juízes online ou *Learning Management Systems* - LMS) limitam-se a analisar submissões finais, ignorando o processo de aprendizagem [Silva et al. 2022].

Este trabalho propõe uma abordagem centrada no desenvolvimento incremental do código, nosso sistema pretende capturar logs em tempo real por meio de um plugin integrado a editores populares (ex.: VS Code¹). Diferentemente de ambientes como Moodle² — que registram apenas interações genéricas (acesso a materiais, participação em fóruns) [Liz-Domínguez et al. 2022] —, nosso sistema captura a evolução dinâmica do código, permitindo:

¹<https://code.visualstudio.com/>.

²<https://moodle.org/>.

- Identificar padrões de dificuldade: Como apresentado por [Gupta et al. 2024], logs de programação revelam lacunas de aprendizagem, mas nossa metodologia amplia essa análise para o contexto de editores de código, onde a interação é mais rica.
- Personalizar o ensino: Rastros temporais (ex.: tempo gasto para digitar o algoritmo) permitem adaptar conteúdos às necessidades individuais.
- Combater plágio: Ao armazenar históricos de estilo de codificação (user profiles), como proposto por [Arabyarmohamady et al. 2012], é possível detectar inconsistências na autoria.

O uso de dispositivos eletrônicos deixa rastros; este plugin desenvolvido oferece uma oportunidade de analisar o processo criativo em tempo real. Cada interação do estudante com o ambiente de desenvolvimento - desde a digitação de palavras reservadas até a execução do código - gera valiosos rastros temporais (*timestamps*) que revelam padrões cognitivos e dificuldades específicas. Este trabalho propõe um sistema de coleta que captura microcomportamentos de programação, incluindo: .

1. Uso de Elementos Sintáticos:

- Tempo entre `if/for/while`
- Erros em `;`, `{}`, `()`

2. Ciclos de Desenvolvimento:

- Tempo de digitação, dificuldade de uso do computador
- Iterações até solução

O objetivo geral deste trabalho é desenvolver e implementar uma extensão para coleta e análise de logs durante o processo de ensino-aprendizagem de programação, com o intuito de fornecer métricas que auxiliem na identificação de dificuldades dos alunos, na personalização do ensino e no aprimoramento das práticas pedagógicas.

1. Desenvolver uma extensão para registro de eventos durante as aulas de programação.
2. Implementar o plugin na IDE utilizada na disciplina, com a anuência da instituição e dos estudantes.
3. Coletar, sincronizar e armazenar dados de eventos gerados durante o experimento.
4. Desenvolver um dataset com os dados coletados, organizados por turma, instituição e disciplina.

A análise vai além da lógica de programação, incluindo a ergonomia do processo de escrita. Dificuldades motoras ou de familiaridade com o teclado manifestam-se em padrões mensuráveis: hesitações antes de símbolos complexos, correções assimétricas em operadores, e discrepâncias entre velocidade de digitação em código versus texto livre. Esses dados permitem:

- Distinguir desafios conceituais de barreiras técnicas
- Adaptar interfaces (ex.: sugerir atalhos para símbolos problemáticos)
- Identificar necessidades de suporte individualizado

Correlação temporal entre teoria aprendida e prática aplicada a relevância dessa proposta está em sua aplicabilidade direta a instituições de ensino técnico e superior. Ao transformar logs em feedback acionável, docentes podem intervir proativamente, e discentes ganham autonomia para corrigir desvios em seu processo de aprendizagem. Além disso, o dataset gerado — inédito por sua granularidade — abre caminho para pesquisas em educação computacional, oferecendo evidências empíricas sobre estratégias pedagógicas eficazes.

2. Trabalhos relacionados

O armazenamento e análise de logs constituem uma área de pesquisa consolidada na engenharia de software, com aplicações em telemetria e monitoramento de sistemas. No âmbito educacional, particularmente no ensino de programação, seu potencial ainda não foi plenamente explorado. Embora essas técnicas sejam comumente empregadas para registrar entregas de atividades e acessos às plataformas, pesquisas recentes mostram sua capacidade de revelar aspectos fundamentais do processo de aprendizagem. Este estudo realiza uma revisão sistemática das abordagens existentes, concentrando-se especificamente em: (1) técnicas de coleta e processamento de logs educacionais e (2) métodos de análise pedagógica capazes de identificar padrões de aprendizagem, dificuldades persistentes e estratégias de resolução de problemas adotadas pelos estudantes.

Bibliotecas virtuais foram consultadas utilizando critérios de busca de palavras-chaves e *strings* de busca, retornando diversos trabalhos. A questão de pesquisa principal investigada neste trabalho é: “Como os logs têm sido utilizados no ensino de programação?”. Além disso, foram definidas subquestões para responder a aspectos específicos sobre a aplicabilidade de cada tecnologia.

A pesquisa também buscou trabalhos elaborados vinculados a Rede de EPCT³ não obtendo resultados para critérios comparativos e ou associativos.

A revisão sistemática da literatura revelou que a maioria dos estudos empíricos sobre análise de aprendizagem em programação utiliza ambientes virtuais de aprendizagem (AVAs) como plataforma primária para coleta de dados. Nesses casos, a instrumentação ocorre predominantemente através de navegadores específicos e ou plugins para o acesso dos estudantes.

A interação dos estudantes com as questões é registrada nos arquivos de log do CodeBench [Pereira et al. 2020], sistema categorizado como um Ambiente de Correção Automática de Códigos (ACAC), também conhecido como Juiz Online (JO). Originalmente desenvolvidos para competições de programação, os JOs foram adaptados para o contexto educacional, tornando-se ferramentas de apoio pedagógico em disciplinas de programação.

Plataformas de aprendizagem (APASs) têm incorporado editores online integrais, permitindo que os estudantes realizem exercícios diretamente no ambiente virtual (Krusche & Seitz, 2018; Mekterović et al., 2020). Essa abordagem proporciona feedback imediato e detalhado, facilitando a identificação de erros e o reforço de habilidades de programação.

³ Rede Federal de Educação Profissional, Científica e Tecnológica, também conhecida por Rede Federal

No contexto de ferramentas especializadas, observa-se o desenvolvimento de Ambientes de Desenvolvimento Integrado (IDEs) adaptados para demandas acadêmicas. Um exemplo notável é o Thonny, IDE projetado para cursos introdutórios de programação, com características que priorizam a experiência do estudante e otimizam a curva de aprendizagem. [Annamaa 2015]

Sistemas computacionais avançados têm potencializado a interação e notificação em ambientes educacionais. Destaque para o PredictCS, que integra análise preditiva com coleta de rastros digitais dos estudantes. No estudo “Analysis of C Programming Performance: A correlational study of novice programmers’ compiler error logs”, evidencia-se a relevância da construção de conjuntos de dados (datasets) para análise de correlações entre características de código, ocorrência de erros sintáticos e acompanhamento do progresso discente [Arguson et al. 2022].

Outra aplicação relevante da geração de logs é a criação de datasets para identificação de padrões de desempenho e visualização de dados educacionais. O avanço nas técnicas de coleta automatizada tem ampliado significativamente a disponibilidade de dados de aprendizagem, viabilizando análises mais robustas. Este trabalho busca consolidar tais dados para aplicações em pesquisas futuras, incluindo iniciativas alinhadas com ciência aberta e práticas de educação baseada em evidências.

2.1. Mapeamento sistemático

Este estudo adota o Mapeamento Sistemático da Literatura (MSL) como metodologia. Essa abordagem permite identificar resultados relevantes por meio de buscas estruturadas em artigos, assegurando uma análise rigorosa e auditável para responder às questões de pesquisa com precisão. [Santana and Magalhães 2024]

Revisões sistemáticas da literatura são altamente recomendáveis para trabalhos, pois avaliam de forma efetiva uma determinada área e entende claramente como sua proposta pode contribuir diante do que já existe publicado. [Neiva and da Silva 2016] As seguintes questões de pesquisa foram elaboradas:

1. Quais são os principais objetivos e aplicações do uso de logs no ensino de programação?
2. Quais ferramentas, métodos e métricas são mais utilizados para capturar, armazenar e analisar logs em ambientes de ensino de programação?

2.2. Critérios de Inclusão

Após a execução das buscas utilizando as *strings* definidas, os artigos identificados foram submetidos a um processo de triagem. Foram adotados os seguintes critérios para inclusão:

1. Relevância temática: O artigo deve abordar explicitamente o uso de logs no ensino de programação, conforme verificado pela análise de títulos e resumos.
2. Revisão sistemática: Trabalhos que se caracterizam como revisões sistemáticas da literatura também foram incluídos, devido à sua contribuição para a compreensão do estado da arte no tema.

2.3. Critérios de Exclusão

Foram excluídos textos que, mesmo contendo a *string* de busca, retornaram resultados relacionados a:

1. *Log-based*: abordagens ou técnicas que utilizam logs como base principal (por exemplo, sistemas de monitoramento ou análise de logs).
2. Uso de logs em outras áreas da educação: aplicações de logs em contextos educacionais que não envolvem diretamente o ensino de programação.
3. *Game-based learning*: trabalhos que utilizam jogos ou mecânicas de jogos como principal estratégia de ensino, sem foco no uso de logs para análise de aprendizagem.

2.4. Estudos sobre Ambientes de Programação e Aprendizagem

As tabelas 1 e 2 resumizam os trabalhos identificados na revisão da literatura, apresentando o ambiente de coleta de logs e o país de origem de cada estudo. Neste contexto, o termo “ambiente” refere-se ao sistema ou plataforma em que os registros de log foram obtidos. Este trabalho adota como ambiente de coleta a configuração IDE/Plugin, utilizando o editor de texto Visual Studio Code (VS Code). Além disso, observa-se uma escassez de pesquisas nessa área no Brasil, particularmente no âmbito das instituições federais de ensino, indicando uma oportunidade para investigações futuras.

Título	Ambiente	País
Analysis of C Programming Performance: A correlational study of novice programmers' compiler error logs	IDE/Online	Filipinas
Programming the Platform University: Learning Analytics, APIs, and Predictive Infrastructures in Higher Education	API/LMS	Austrália
Analyzing git log in an code-quality-aware automated programmement system: a case study	APAS/LMS/Devops	Vietnã
Using learning analytics in the Amazonas: understanding students' behaviour in introductory programming	OJ/IDE	Brasil
Logging Practices in Software Engineering: A Systematic Mapping Study	SMS/DevOps	China/Austrália
Previsão de indicadores de dificuldade de questões de programação a partir de métricas do código de solução	IDE-Online/OJ	Brasil
Where Do Developers Log? An Empirical Study on Logging Practices in Industry	Survey	China/EUA
Logging Practices in Software Engineering: A Systematic Mapping Study	SMS/DevOps	Canadá
Identification of Student Programming Patterns through Clickstream Data	OCP/Mining	Índia

Tabela 1. Estudos sobre Ambientes de Programação e Aprendizagem (Parte 1)

Título	Ambiente	País
An Integrated Program Analysis Framework for Graduate Courses in Programming Languages and Software Engineering	Framework/Analysis	Índia
Personalizing Computer Science Education by Leveraging Multimodal Learning Analytics	Framework/Analysis	Irlanda/EUA
Introducing Thonny, a Python IDE for Learning Programming	IDE	Estônia
Learning log-based automatic group formation: system design and classroom implementation study	LMS/Analysis	Japão
Personalizing Computer Science Education by Leveraging Multimodal Learning Analytics	Framework/Analysis	Irlanda/EUA

Tabela 2. Estudos sobre Ambientes de Programação e Aprendizagem - (Parte 2)

A pesquisa foi realizada em cinco bibliotecas virtuais, utilizando como critérios de busca palavras-chave e *strings* de busca com operadores lógicos *AND* e *OR*. Os repositórios de pesquisa utilizados são apresentados na Tabela 3.

Repositório	Resultados
Scopus	23
IEEE Xplore	2
SOL SBC	2
ACM	127
WILEY	9

Tabela 3. Lista de repositórios de pesquisa utilizados no estudo.

As combinações de palavras-chave e operadores utilizadas na pesquisa são apresentadas na Tabela 4.

Palavras-chave e Operadores
programming AND teaching AND log AND NOT (log-based) AND NOT (metrics) AND systematic AND review
programming AND teaching AND log AND NOT (log-based) AND (integrate AND development AND environment) AND NOT (visual AND programming) AND NOT (web AND application)
telemetry AND education AND programming AND log
ontology AND education AND programming AND log AND dataset
artefact AND cognitive AND learning
background AND teaching AND log AND NOT (log-based) AND (systematic AND review)
log AND (“teaching programming”OR “learning programming”) AND (“IDE”OR “integrated development environment”)) NOT (“log-based”OR “lms”OR “learning management system”
(log AND (“teaching programming”OR “learning programming”) AND (“IDE”OR “integrated development environment”)) NOT (“log-based”OR “lms”OR “learning management system”OR “game-based”OR “game based”OR “gamification”)

Tabela 4. Combinações de palavras-chave e operadores utilizados na pesquisa.

2.5. RESPOSTAS PARA QP1: QUAIS SÃO OS PRINCIPAIS OBJETIVOS E APLICAÇÕES DO USO DE LOGS NO ENSINO DE PROGRAMAÇÃO?

Conforme o framework proposto por [Gu et al. 2022], o processo de coleta de logs deve ser guiado por quatro dimensões fundamentais: (1) propósito da coleta, (2) localização dos pontos de instrumentação, (3) especificação dos eventos registrados, e (4) metodologia de armazenamento e análise.

A abordagem feita por [Silva et al. 2022] busca “prever indicadores de dificuldade de exercícios de programação a partir de métricas de complexidade do código dos instrutores. Identificando registros de logs de JOs e relacionando esforço e dificuldade dos estudantes durante o desenvolvimento do código de solução”.

De acordo com [Gupta et al. 2024], “os dados de cliques (clickstream) e logs de atividades dos estudantes em plataformas de programação podem ser processados para extrair informações relevantes, como padrões de programação e relações entre características do código e o desempenho dos alunos. Essa análise permite que os instrutores identifiquem previamente estudantes com dificuldades ou baixo desempenho, ajustando suas metodologias de ensino e oferecendo suporte direcionado. Além disso, ao acompanhar a evolução do estilo de codificação dos alunos, os educadores podem avaliar a eficácia de suas estratégias pedagógicas e garantir que nenhum estudante seja deixado para trás em um mundo tecnologicamente avançado”.

Nesse âmbito, a identificação e prevenção do plágio em atividades práticas configura-se como uma preocupação pedagógica relevante. Os registros de log (logs) emergem como ferramenta estratégica nesse contexto, oferecendo suporte à avaliação discente por meio de:

- **Rastreabilidade:** Monitoramento cronológico do processo de desenvolvimento
- **Autenticidade:** Verificação da originalidade do código produzido
- **Análise comparativa:** Identificação de padrões no desenvolvimento das atividades propostas.

Como argumenta [Arabyarmohamady et al. 2012], características extraídas de cada código, como o estilo de programação, são armazenadas em um histórico (*user profile*). Esses dados permitem identificar mudanças no estilo de codificação e detectar se um código foi realmente desenvolvido pelo autor declarado ou se foi escrito por outra pessoa com um estilo diferente. Dessa forma, os logs não apenas auxiliam na identificação de plágio, mas também contribuem para uma avaliação mais precisa e personalizada do aprendizado dos estudantes.

Outra aplicação relevante para a geração de logs é a criação de datasets, como demonstrado no trabalho de [Umezawa et al. 2020]. O estudo detecta diversos elementos em códigos-fonte, como declarações de variáveis, funções, estruturas condicionais, laços de repetição, operações de leitura e escrita, espaçamento, pontuação, expressões lógicas e matemáticas, além de comentários. Esses dados são organizados em tabelas para análise.

3. Metodologia

Esta pesquisa tem o intuito de entender o processo de construção de algoritmos, as ferramentas e técnicas de engenharia de software utilizados no ensino, por meio de logs cria-

dos e armazenados no processo de ensino-aprendizagem em um IF da região Amazônica. Com esta coleta teremos informações quali-quantitativas sobre a produção de códigos.

Em termos de formulação de modelos com possibilidades de aplicação no ensino de programação e desenvolvimento de soluções para problemas locais. O estabelecimento de tal estrutura, permitiria o compartilhamento de conhecimentos de maneira cumulativa, podendo ser utilizado e aprimorado continuamente por sucessivas turmas de alunos, de diferentes cursos.

Nesse contexto, conforme destacado por [Qian and Lehman 2017], “esforços para aprimorar o ensino em ciência da computação estão em andamento, e os docentes enfrentam desafios significativos em disciplinas introdutórias de programação, visando facilitar a aprendizagem dos estudantes”. Nos cursos técnicos em desenvolvimento de software ofertados pela Rede de EPCT, disciplinas como Introdução a Algoritmos, Programação Básica, Estrutura de Dados e Programação Orientada a Objetos são frequentemente associadas a altas taxas de dificuldade e evasão. Além dos obstáculos cognitivos, os alunos frequentemente lidam com questões emocionais, sentindo-se incapazes de dominar os conceitos e, conseqüentemente, desenvolvendo a crença de que são inaptos para a área de informática como um todo. A aplicação da métrica proposta neste estudo permitiria uma análise quali-quantitativa do desempenho algorítmico e do uso do computador, oferecendo subsídios valiosos para reduzir esses problemas.

O papel central dos IFs é promover o desenvolvimento socioeconômico local e regional, por meio de pesquisas aplicadas e da criação de soluções técnicas e tecnológicas alinhadas às demandas da comunidade, fortalecendo os arranjos produtivos locais [Rodrigues and Gava 2016].

Sendo assim, pode-se interpretar esta característica dos IFs como um potencializador do uso de plataformas de códigos, estimulando os alunos a se dedicarem ao desenvolvimento de soluções para problemas locais, ao mesmo tempo permitindo a interação com outras experiências e aportes colaborativos diversos.

A aplicação de princípios de Engenharia de Software aliada à análise de métricas de uso de ambientes de desenvolvimento integrado (IDE) possibilitará avaliações mais robustas. A opção por uma IDE amplamente adotada no setor profissional busca uniformizar as ferramentas utilizadas no contexto acadêmico, eliminando a dissonância entre disciplinas. A solução proposta consiste em uma extensão de operação transparente, executando-se em segundo plano sem comprometer a experiência de usuários finais (discentes e docentes) durante suas atividades de desenvolvimento.

A análise dos dados coletados pelo plugin será realizada em três etapas principais:

1. Coleta e Pré-processamento

1.1. Registro de Logs

- Timestamp (data/hora de cada ação)
- Trechos de código editados
- Padrões de estilo (ex.: nomenclatura de variáveis, estrutura de funções)
- Eventos como compilações, erros e correções

2.2. Filtragem e Anonimização

- Dados sensíveis (como nomes ou matrículas discentes) são substituídos por identificadores únicos, garantindo a privacidade.
2. Análise de Autoria e Detecção de Plágio
 - 1.1. Perfil do estudante (User Profile)
 - Frequência de commits
 - Uso de estruturas específicas (ex.: loops, funções)
 3. Geração de Dados Educacionais a partir de eventos
 - Dataset para Pesquisa
 - Eficácia de metodologias de ensino
 - Correlação entre padrões de codificação e desempenho acadêmico

Apesar do potencial dos logs para corroborar o ensino de programação, ainda há desafios significativos na coleta, organização e interpretação desses dados. Muitas vezes, as ferramentas existentes não são adaptadas ao contexto educacional. Todo projeto computacional voltado ao ensino apresenta possíveis riscos. No caso desta proposta, os principais riscos identificados são:

1. Falha técnica no funcionamento do plugin
 - Possibilidade de o plugin não executar conforme o esperado devido a:
 - Incompatibilidade com versões desatualizadas do editor de texto principal;
 - Interferência de antivírus ou firewalls;
 - Problemas de conexão com o repositório remoto;
 - Erros durante o armazenamento dos logs.
2. Recusa do docente em utilizar a ferramenta
 - O professor responsável pode, por motivos técnicos ou pessoais, optar por não instalar o plugin nos computadores dos estudantes.
3. Negativa do discente em participar da coleta de dados
 - Embora a ferramenta colete apenas eventos relacionados ao ambiente de texto (sem armazenar dados sensíveis ou interferir em outras atividades), o estudante pode optar por não contribuir com os logs.
4. Exposição inadvertida de dados
 - Apesar de o sistema não registrar informações pessoais ou sensíveis, há um risco mínimo de que eventos acadêmicos (como interações no editor) sejam visualizados por outros participantes sem autorização.

4. Considerações Finais

Este artigo buscou informações sobre o uso de logs no ensino de programação focado nas Instituições Federais de Educação, Ciência e Tecnologia. Nesse sentido, foi buscado informações sobre aplicações de logs no aprendizado e também trabalhos que relatam a pesquisa e a aplicação do tema. Fora realizada uma revisão da literatura nos repositórios de artigos.

Verificou-se que as ferramentas e ambientes de programação variam nas instituições de ensino, sendo assim, uma extensão para um editor de texto de código aberto poderia auxiliar na coleta de logs. Testes e execuções do projeto nos ambientes acadêmicos serão realizados para que o plugin seja melhorado e posteriormente divulgado e compartilhado com outras instituições e docentes.

O plugin pode preencher uma lacuna nas IDEs utilizadas nos ambientes acadêmico, coletando e armazenando logs no desenvolvimento de códigos, contudo, estes rastros talvez não possuam um padrão ou reflitam no desempenho e no aprendizado.

Uma base de dados com esses rastros acadêmicos comprovaria a finalidade deste projeto, além de proporcionar outras pesquisas e aplicações, como “big data” na educação, avaliações pedagógicas quantitativas e identificações de pontos de aprendizado e dúvidas dos estudantes.

Referências

- Annamaa, A. (2015). Introducing thonny, a python ide for learning programming. In *Proceedings of the 15th koli calling conference on computing education research*, pages 117–121.
- Arabyarmohamady, S., Moradi, H., and Asadpour, M. (2012). A coding style-based plagiarism detection. In *Proceedings of 2012 International Conference on Interactive Mobile and Computer Aided Learning (IMCL)*, pages 180–186. IEEE.
- Arguson, A., Malasaga, E., Aquino, A., Cheng, R., Ambat, S., and Tejuco, H. (2022). Analysis of c programming performance: A correlational study of novice programmers’ compiler error logs. In *2022 IEEE 14th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM)*, pages 1–5. IEEE.
- Gu, S., Rong, G., Zhang, H., and Shen, H. (2022). Logging practices in software engineering: A systematic mapping study. *IEEE Transactions on Software Engineering*, 49(2):902–923.
- Gupta, A., Jindal, M., and Goyal, A. (2024). Identification of student programming patterns through clickstream data. In *2024 IEEE International Conference on Computing, Power and Communication Technologies (IC2PCT)*, volume 5, pages 1153–1158. IEEE.
- Liz-Domínguez, M., Llamas-Nistal, M., Caeiro-Rodríguez, M., and Mikic-Fonte, F. (2022). Lms logs and student performance: The influence of retaking a course. In *2022 IEEE Global Engineering Education Conference (EDUCON)*, pages 1970–1974. IEEE.
- Neiva, F. W. and da Silva, R. d. S. (2016). Revisão sistemática da literatura em ciência da computação um guia prático. *Universidade Federal de Juiz de Fora*.
- Pereira, F. D., Oliveira, E. H., Oliveira, D. B., Cristea, A. I., Carvalho, L. S., Fonseca, S. C., Toda, A., and Isotani, S. (2020). Using learning analytics in the amazonas: understanding students’ behaviour in introductory programming. *British journal of educational technology*, 51(4):955–972.
- Qian, Y. and Lehman, J. (2017). Students’ misconceptions and other difficulties in introductory programming: A literature review. *ACM Transactions on Computing Education (TOCE)*, 18(1):1–24.
- Rodrigues, F. C. R. and Gava, R. (2016). Universidades federais no estado de minas gerais: Um estudo comparativo. *REAd — Porto Alegre — Edição 83*.

- Santana, F. P. and Magalhães, L. C. (2024). Aplicações do processamento de linguagem natural no ambiente educacional: Uma revisão sistemática da literatura. *Revista Foco*, 17(1):e3921–e3921.
- Silva, E. S., Carvalho, L. S., de Oliveira, D. B., Oliveira, E. H., Lauschner, T., de Lima, M. A., and Pereira, F. D. (2022). Previsão de indicadores de dificuldade de questões de programação a partir de métricas do código de solução. In *Anais do XXXIII Simpósio Brasileiro de Informática na Educação*, pages 859–870. SBC.
- Umezawa, K., Nakazawa, M., Kobayashi, M., Ishii, Y., Nakano, M., and Hirasawa, S. (2020). Analysis of logic errors utilizing a large amount of file history during programming learning. In *2020 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALE)*, pages 630–634. IEEE.